PHYS 311 — Lab 9
Multi-Body Browser Simulation

Alex Bebb

November 2026

Live simulation:
`https://agbutk1993.github.io/phys311/labs/lab09/`

Code location: `phys311/labs/lab09/`

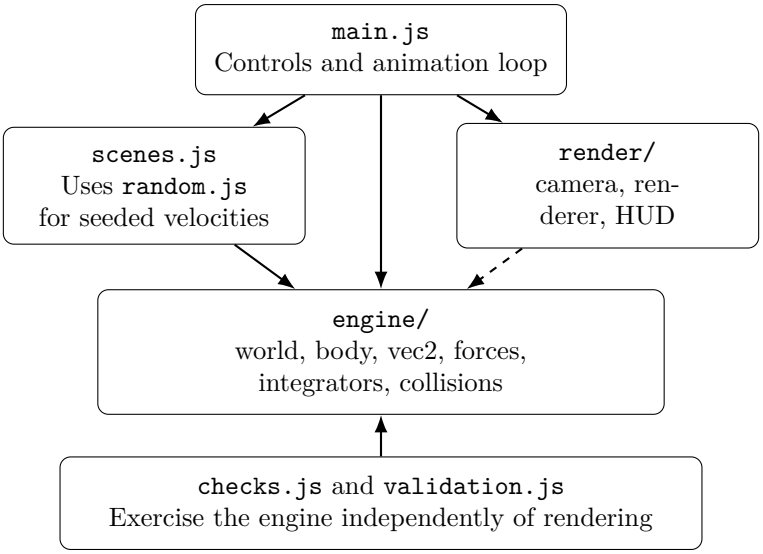
Part A: Engine Organization

I used the same JavaScript physics engine for a box of colliding balls, a gravitational system, and a spring chain. The scenes supplied their own bodies, forces, and boundary settings. The engine calculated the motion, while the rendering code displayed the results.

The engine contained six modules: `vec2.js`, `body.js`, `forces.js`, `integrators.js`, `collisions.js`, and `world.js`. The rendering folder contained `camera.js`, `renderer.js`, and `hud.js`.

Module Dependency Diagram

The diagram groups related modules for readability. Solid arrows mean "uses." The dashed arrow means the rendering modules read the engine state; they do not update positions or velocities.



Within the engine, `body.js` uses `vec2.js`. `world.js` uses the vector, integrator, and collision modules, and calls the force objects supplied by the scene. The renderer uses the camera to convert physical coordinates to screen coordinates and recognizes spring objects to draw their connections.

The main module also uses `performance.js` for timing averages and `replay.js` for binary state comparisons.

Time Stepping

Each `World.step` cleared accumulated forces, applied the force objects, advanced the bodies with the selected integrator, resolved enabled collisions, and advanced simulation time. Velocity Verlet also recalculated forces at the updated positions before completing the velocity update.

The physics time step was fixed at

$$\Delta t = \frac{1}{240} \text{ s}.$$

The browser could perform several physics steps before drawing a frame. This separated the physics time step from the browser's changing frame intervals.

Part B: Collisions and Validation

Separating-Guard Demonstration

With the separating guard disabled, two overlapping balls that were already moving apart received an impulse that reversed their velocities, sending them back toward each other. Restoring the `vn > 0` guard prevented this unnecessary impulse while still allowing overlap correction. This removes a mechanism that can produce repeated bouncing or jitter at contact.

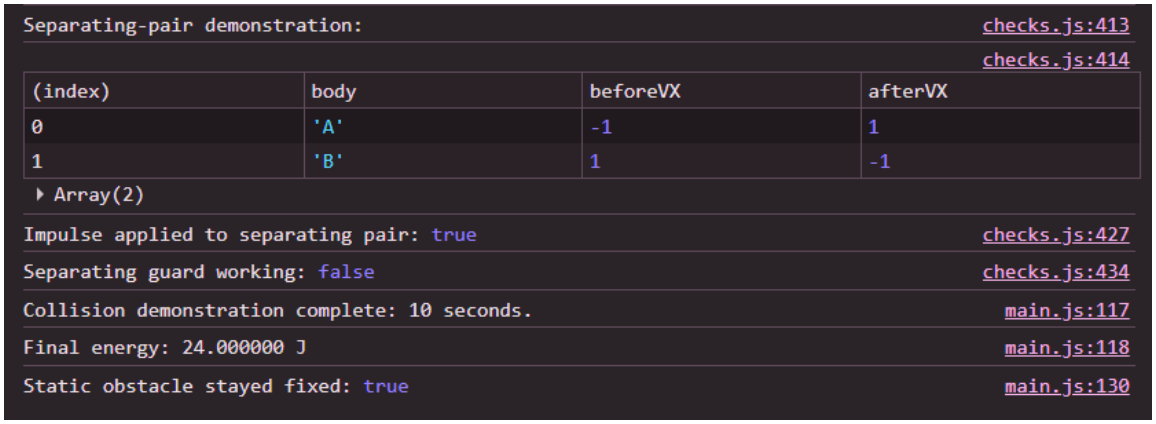


Figure 1: Guard disabled: the separating pair incorrectly received an impulse, reversing its velocities.

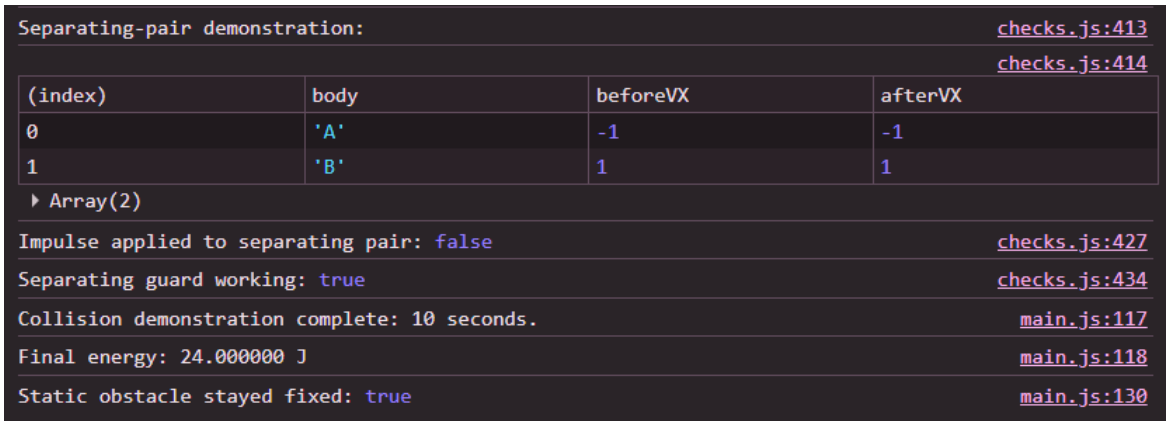


Figure 2: Guard enabled: the velocities remained unchanged and no impulse was applied.

The collision implementation also used inverse-mass-weighted overlap correction and restitution impulses. A static-obstacle check confirmed that a moving body rebounded while the obstacle stayed fixed.

HUD

The HUD displayed time, body count, total energy, percentage energy change, total momentum magnitude, FPS, and physics and rendering times. Energy change turned red when its magnitude exceeded 1%. For inelastic collisions, a red energy-change value can represent expected physical energy loss rather than a numerical error.

Conservation Tests

I ran four tests with 20 balls for 60 simulated seconds, using $\Delta t = 1/240$ s and no gravity or other applied forces. The validation tests used the same initial conditions as one another; these differed from the visible box scene.

There is an important distinction concerning the box walls: fixed walls exert external impulses on the balls. Therefore, the balls alone do not form an isolated system, even without gravity. I tested energy behavior with walls enabled and also ran walls-off tests to check conservation of the total momentum vector.

Here the fractional energy change is

$$\delta_E = \frac{E(60) - E(0)}{|E(0)|}.$$

It is a fraction, not a percentage.

Walls	e	$E(0)$ (J)	$E(60)$ (J)	δ_E
On	1.0	41.853482788	41.853482788	$-5.093072 \times 10^{-16}$
On	0.8	41.853482788	2.260716554	-9.459850×10^{-1}
Off	1.0	41.853482788	41.853482788	$-5.093072 \times 10^{-16}$
Off	0.8	41.853482788	29.738594472	-2.894595×10^{-1}

Table 1: Initial and final kinetic energies. With no force potentials in these tests, kinetic energy was the total mechanical energy.

Walls	e	$\lvert\mathbf{P}(0)\rvert$ (kg m/s)	$\lvert\mathbf{P}(60)\rvert$ (kg m/s)
On	1.0	8.022388727	11.389043173
On	0.8	8.022388727	3.585494633
Off	1.0	8.022388727	8.022388727
Off	0.8	8.022388727	8.022388727

Table 2: Momentum magnitudes. The automated walls-off checks also compared the full momentum vectors throughout the runs.

The elastic tests conserved energy within floating-point rounding. The inelastic tests showed no stepwise energy increase beyond the numerical tolerance. Both walls-off tests conserved total momentum within the tolerance, and all four validation cases passed.

In an isolated collision, equal and opposite impulses conserve total momentum even when $e < 1$. Kinetic energy can decrease because some energy goes into deformation, heat, or other internal processes that the model does not explicitly track. With fixed walls present, the walls also exchange momentum with the balls.

Part C: Three Scenes

Colliding Balls

The box scene supported adjustable restitution and body count. Elastic collisions preserved kinetic energy, while inelastic collisions reduced it. The controls also allowed the integrator to be changed during an ordinary run.

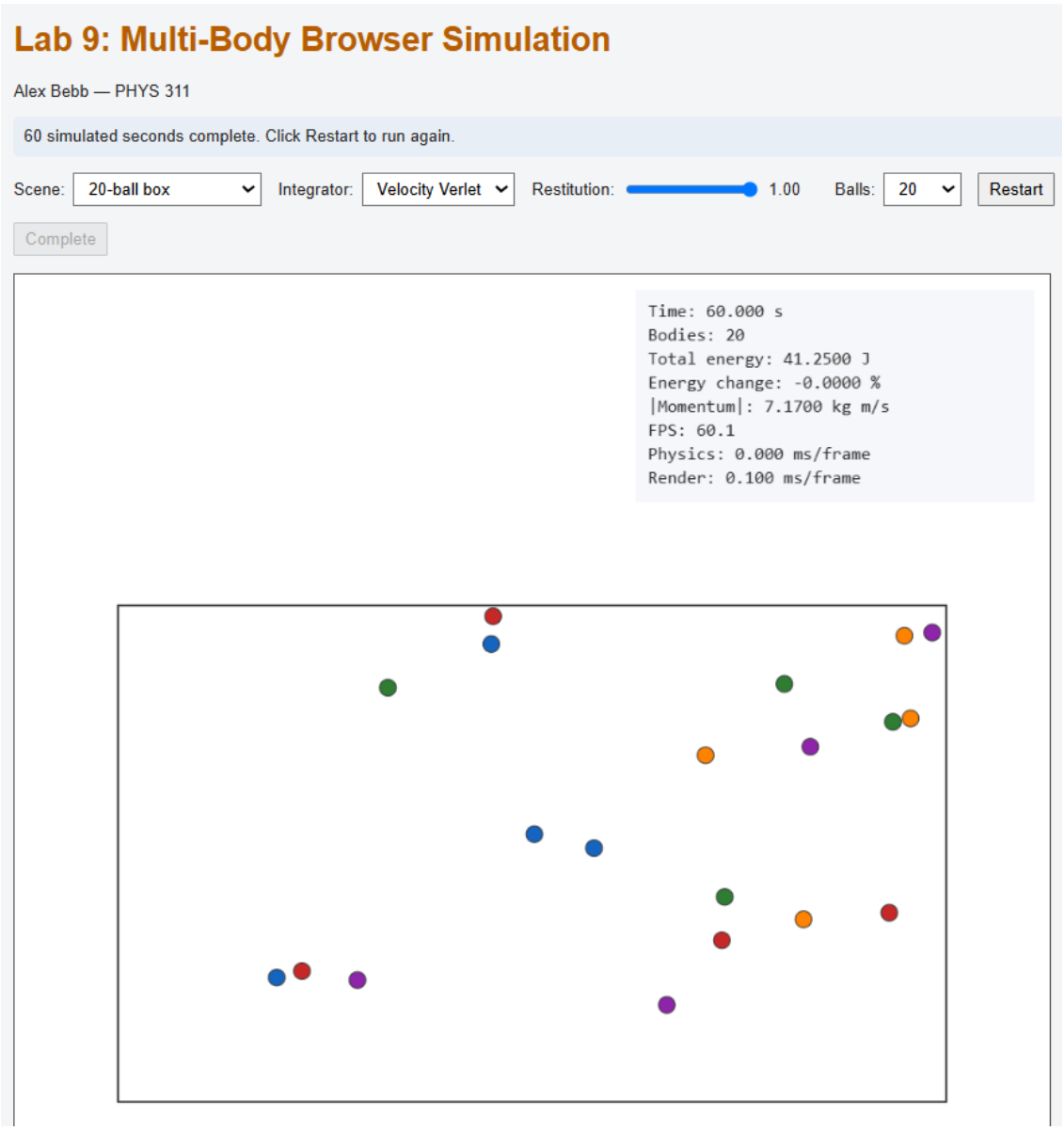


Figure 3: The 20-ball elastic box scene after 60 simulated seconds. The stopped physics loop explains the zero physics time shown here; performance was measured separately while the simulation was running.

Gravitational Orbits

One body of mass 100 interacted with two bodies of mass 0.1 each. All three bodies were free to move. I used $G = 1$ in the simulation's unit system and a softening length of 0.05. The initial state was shifted into the center-of-mass frame.

With Velocity Verlet, the maximum relative energy error over 60 seconds was 8.979103×10^{-8}, or approximately 0.00000898%. The maximum momentum-vector change was 1.588640×10^{-14} kg m/s. The trails remained approximately circular during the tested interval.

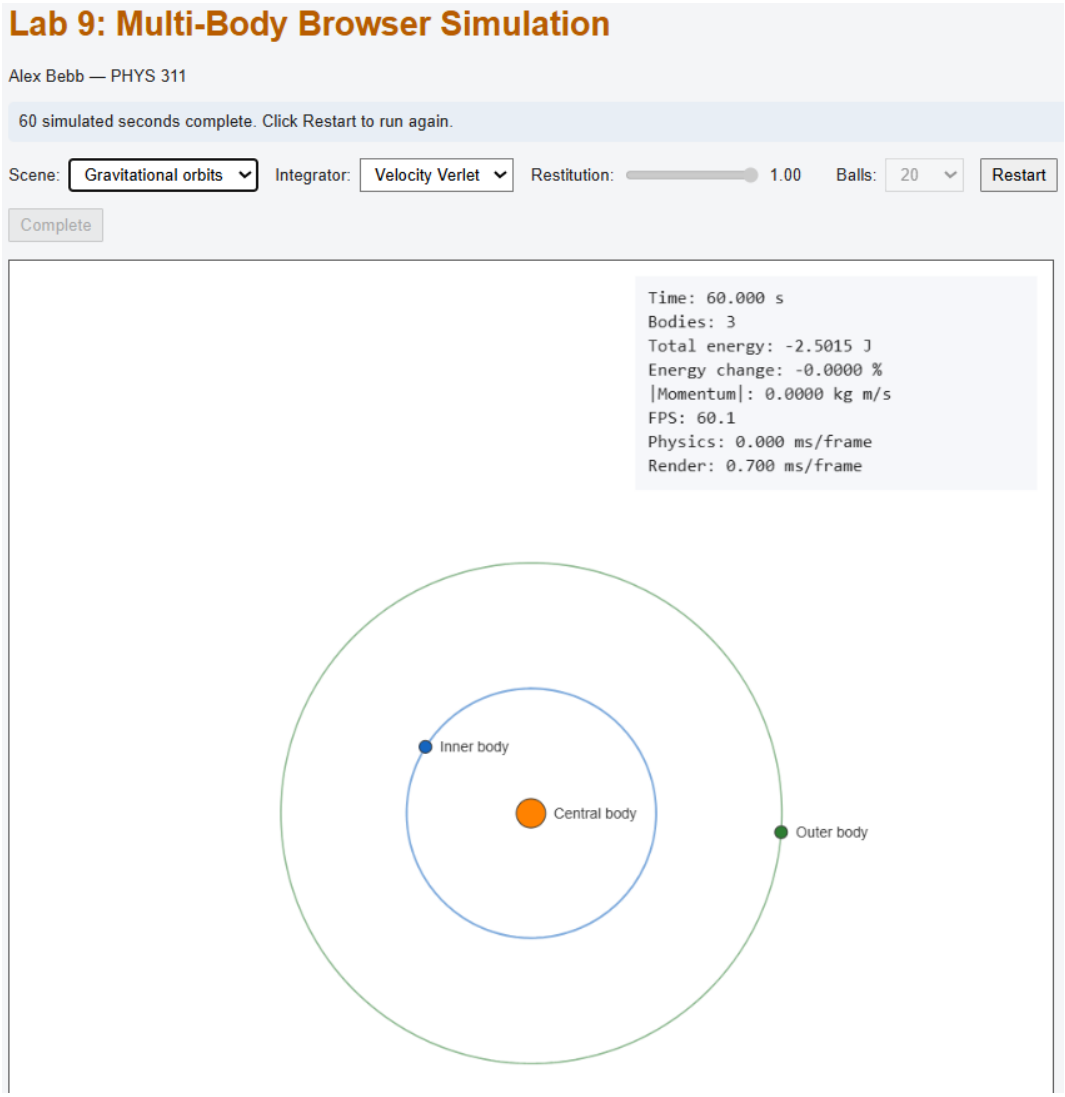


Figure 4: The gravitational scene with orbital trails and the HUD. The central body's response was small because it was much heavier than either orbiting body.

Spring Chain

Four unit masses were connected to a fixed anchor by springs with stiffness 10 N/m and rest length 1.5 m. Small initial velocities and an offset of the final mass started the motion. Gravity, damping, and collisions were disabled.

The maximum relative energy error over 60 seconds with Velocity Verlet was 3.581261×10^{-5}, or approximately 0.00358%. The anchor could change the moving masses' total momentum, but it did no work because its position remained fixed.

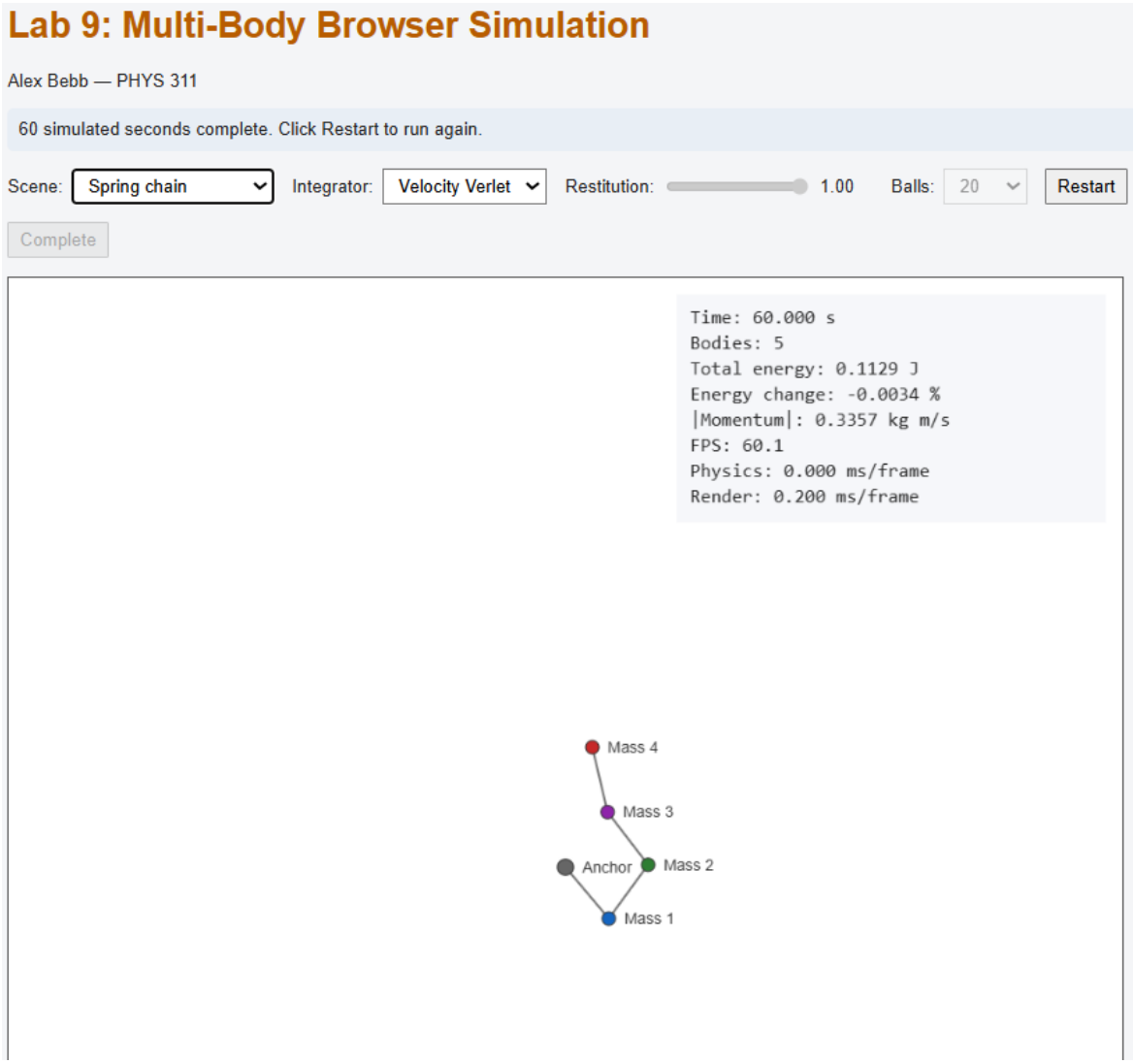


Figure 5: The spring chain after 60 seconds. The HUD counts five bodies: four moving masses and the fixed anchor.

Performance Scaling

I measured the elastic box scene using Velocity Verlet and the fixed time step of 1/240 s. Each measurement used a two-second warm-up followed by a ten-second average.

Balls	Physics (ms/frame)	Render (ms/frame)	FPS	Simulation rate (s/s)
20	0.066	0.389	60.1	1.000
50	0.300	0.494	60.1	1.000
100	1.122	0.673	60.1	1.000
200	3.134	0.713	60.1	1.000

Table 3: Performance measured on my computer.

All tested counts maintained approximately 60 FPS and kept up with real time. The largest tested count was 200 balls; this did not establish the actual maximum capacity.

Physics became the larger measured cost at higher body counts. The collision routine checked every pair, requiring

$$\frac{N(N-1)}{2}$$

pair checks per physics step. This increased from 190 pairs for 20 balls to 19,900 pairs for 200 balls. The approximately quadratic growth makes pair checking a likely scaling bottleneck.

Physics timing included the conservation diagnostics. Render timing covered drawing calls and HUD work rather than separately measuring GPU execution.

An initial 200-ball measurement showed a much lower frame rate. I repeated it with the tab kept visible and the controls unchanged, obtaining the result reported above. The exact cause of the earlier slowdown was not established.

Changes Needed for the Scenes

Once the base engine was working, the three Part C scenes did not require separate integrators or scene-specific changes to the physics-stepping code. They used different combinations of the existing bodies, forces, collision settings, and boundaries.

The orbit scene required renderer-owned trail history and different camera settings. The spring scene required drawing connections between spring endpoints. Scene switching also required resetting time, diagnostic baselines, and trails. These changes belonged to the scene setup, rendering, and browser controls.

The models still have limitations. Discrete collision checks can miss sufficiently fast contacts, and sequential pair resolution approximates simultaneous collisions. The spring chain has no bending resistance or self-collisions. Small energy errors support the tested simulations, but do not prove every trajectory is accurate.

Bonus: Deterministic Replay

I added a seeded pseudo-random generator for the box scene's starting velocities. The Seeded run button saved the settings and final state. Replay rebuilt the scene from the same settings and seed.

The test used 20 balls, seed 31109, Velocity Verlet, restitution 1, and $\Delta t = 1/240$ s. Each run completed 14,400 physics steps, representing 60 simulated seconds.

The comparison encoded the final physical state and simulation metadata as Float64 bytes. All 1,960 compared bytes matched. Bodies were matched by their order in the scene; automatically assigned IDs were excluded because their counter increased when new bodies were created.

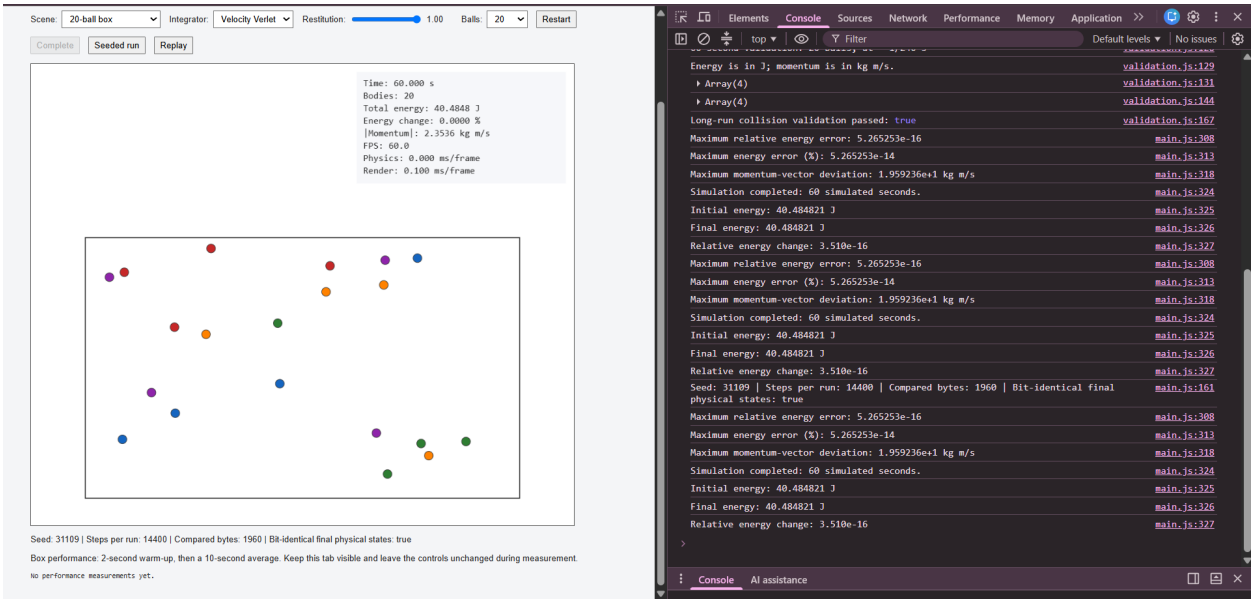


Figure 6: The replay comparison reported bit-identical final physical states after both runs completed.

This demonstrated reproducibility for the tested configuration in the same browser. It did not establish physical accuracy by itself: a simulation could repeat the same mistake consistently. The separate collision and conservation checks were still needed, and this test does not guarantee identical results across all browsers or platforms.